

Structured Training Program: Advance C Programming

Week 2 – Signals and Threads

1. Write a program to define 3 different threads with the following purpose where N is the input

- Thread A - To run a loop and return the sum of first N prime numbers.
- Thread B and C – should run in parallel. One prints “Thread 1 running every 2 second”, and the other prints “Thread 2 running every 3 second”

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <math.h>
#include <stdlib.h>

void* sumOfNPrimes(void* N);

void* threadRunFor100Seconds(void* N);

int main() {
    pthread_t thread1, thread2, thread3;
    int n1 = 10;
    // n2 and n3 are sleep time for thread 2 and thread 3
    int n2 = 2;
    int n3 = 3;

    pthread_create(&thread1, NULL, sumOfNPrimes, &n1);
    pthread_join(thread1, NULL);

    //fixed: initially was in sequential exection along with thread1
    pthread_create(&thread2, NULL, threadRunFor100Seconds, &n2);
    pthread_create(&thread3, NULL, threadRunFor100Seconds, &n3);
    pthread_join(thread2, NULL);
    pthread_join(thread3, NULL);

    return 0;
}

void* sumOfNPrimes(void* N) {
    int n = *(int*)N;
    if (n < 1) {
        printf("N must be greater than 0.\n");
        return NULL;
    }
    // Upper bound
    int limit = (n < 6) ? 15 : (int)(n * (log(n) + log(log(n))));
    int* primes = (int*)malloc(limit * sizeof(int));
```

```

primes[0] = 2;
int count = 1;
long long sum = 2;

printf("Prime 1: 2\n");

for (int num = 3; count < n; num += 2) {
    int isPrime = 1;
    for (int i = 0; i < count; i++) {
        if (primes[i] * primes[i] > num) {
            break;
        }
        if (num % primes[i] == 0) {
            isPrime = 0;
            break;
        }
    }

    if (isPrime) {
        primes[count++] = num;
        sum += num;
        printf("Prime %d: %d\n", count, num);
    }
}

printf("\nTotal Sum = %lld for n primes is %d\n\n", sum, n);

free(primes);
return NULL;
}

void* threadRunFor100Seconds(void* N) {
    int interval = *(int*)N;
    int elapsed = 0;

    while (elapsed < 100) {
        int chunk = interval;

        if (elapsed + chunk > 100) {
            chunk = 100 - elapsed;
        }

        sleep(chunk);
        elapsed += chunk;
        printf("Thread running every %d seconds.\n", interval);
    }

    printf("Thread finished after 100 seconds.\n");
    return NULL;
}

```

Output

```

mysterychap@mysteryChap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$ gcc thread.c -o thread.out -pthread -lm
mysterychap@mysteryChap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$ ./thread.out

mysterychap@mysteryChap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$

```

2. In the above program,

- Add signal handler for SIGINT (etc) and prevent termination.
- Convert the above threads to individual functions and note down the time taken and the flow of execution.

Code:

- Add signal handler for SIGINT (etc) and prevent termination.

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <pthread.h>
#include <unistd.h>
#include <math.h>

void* findNPrimeNumbers(void* N);
void* threadRunFor100Seconds(void* N);
void handleTermination(int sig);

int main() {

    //func call to handle signal
    signal(SIGINT, handleTermination);

    int n1 = 10, n2 = 2, n3 = 3;

```

```
pthread_t thread1, thread2, thread3;

pthread_create(&thread1, NULL, findNPrimeNumbers, &n1);
pthread_join(thread1, NULL);

//fixed: initially was in sequential exection along with thread1
pthread_create(&thread2, NULL, threadRunFor100Seconds, &n2);
pthread_create(&thread3, NULL, threadRunFor100Seconds, &n3);
pthread_join(thread2, NULL);
pthread_join(thread3, NULL);
}
```

```
void* findNPrimeNumbers(void* N) {
    int n = *(int*) N;

    int limit = n < 6 ? 15 : (int) (n * (log(n) + log(log(n))));
    int* primes = (int*)malloc(limit * sizeof(int));
    primes[0] = 2;
    int count = 1;
    long long sum = 2;

    printf("Prime 1: 2\n");

    for (int num = 3; count < n; num += 2) {
        int isPrime = 1;
        for (int i = 0; i < count; i++) {
            if (primes[i] * primes[i] > num) {
                break;
            }
            if (num % primes[i] == 0) {
                isPrime = 0;
                break;
            }
        }

        if (isPrime) {
            primes[count++] = num;
            sum += num;
            printf("Prime %d: %d\n", count, num);
        }
    }

    printf("\nTotal Sum = %lld for n primes is %d\n\n", sum, n);

    free(primes);
    return NULL;
}
```

```
void* threadRunFor100Seconds(void* N) {
    int interval = *(int*)N;
    int elapsed = 0;
```

```

while (elapsed < 100) {
    int chunk = interval;

    if (elapsed + chunk > 100) {
        chunk = 100 - elapsed;
    }

    sleep(chunk);
    elapsed += chunk;
    printf("Thread running every %d seconds.\n", interval);
}

printf("Thread finished after 100 seconds.\n");
return NULL;
}

void handleTermination(int sig) {
    printf("\nTermination handled; currently inside function: %s\n", __func__);
}

```

Output:

```

mysterychap
mysterychap@mysteryChap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$ gcc signals.c -o signals.out -pthread -lm
mysterychap@mysteryChap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$ ./signals.out
Prime 1: 2
Prime 2: 3
Prime 3: 5
Prime 4: 7
Prime 5: 11
Prime 6: 13
Prime 7: 17
Prime 8: 19
Prime 9: 23
Prime 10: 29

Total Sum = 129 for n primes is 10

Thread running every 2 seconds.
Thread running every 3 seconds.
Thread running every 2 seconds.
Thread running every 3 seconds.
Thread running every 2 seconds.
^C
Termination handled; currently inside function: handleTermination
Thread running every 2 seconds.
Thread running every 3 seconds.
^C
Termination handled; currently inside function: handleTermination
Thread running every 2 seconds.
Thread running every 2 seconds.
Thread running every 3 seconds.
Thread running every 2 seconds.
^Z
[1]+  Stopped                  ./signals.out
mysterychap@mysteryChap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$

```

b) Convert the above threads to individual functions and note down the time taken and the flow of execution.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <pthread.h>
#include <unistd.h>
#include <math.h>
#include <time.h>

long long primeSum = 0;
void* findNPrimeNumbers(void* N);

int main() {
    int n;
    printf("Enter the N value: ");
    scanf("%d", &n);
    pthread_t thread;
    clock_t start1 = clock();
    findNPrimeNumbers(&n);
    clock_t end1 = clock();
    double functionTime = (double)(end1 - start1) / CLOCKS_PER_SEC;
    long long normalSum = primeSum;
    clock_t start2 = clock();
    pthread_create(&thread, NULL, findNPrimeNumbers, &n);
    pthread_join(thread, NULL);
    clock_t end2 = clock();
    double threadTime = (double)(end2 - start2) / CLOCKS_PER_SEC;
    long long threadSum = primeSum;
    printf("Prime Sum: %lld\n", normalSum);
    printf("Normal Function Time: %.6f seconds\n", functionTime);
    printf("Prime Sum: %lld\n", threadSum);
    printf("Thread Execution Time: %.6f seconds\n", threadTime);
    return 0;
}

void* findNPrimeNumbers(void* N) {
    int n = *(int*)N;
    int limit = n < 6 ? 15 : (int)(n * (log(n) + log(log(n))));
    int* primes = (int*)malloc(limit * sizeof(int));
    primes[0] = 2;
    int count = 1;
    long long sum = 2;
    for (int num = 3; count < n; num += 2) {
        int isPrime = 1;
        for (int i = 0; i < count; i++) {
            if (primes[i] * primes[i] > num)
                break;
            if (num % primes[i] == 0) {
                isPrime = 0;
            }
        }
        if (isPrime) {
            primes[count] = num;
            count++;
            sum += num;
        }
    }
    primeSum = sum;
    return NULL;
}
```

```

        break;
    }
}
if (isPrime) {
    primes[count++] = num;
    sum += num;
}
}
primeSum = sum;
free(primes);
return NULL;
}

```

Output:

```

mysterychap@mysterychap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$ gcc calculatettime.c -o calculatettime.out -pthread -lm
mysterychap@mysterychap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$ ./calculatettime.out
Enter the N value: 300
Prime Sum: 271061
Normal Function Time: 0.000106 seconds
Thread Execution Time: 0.000259 seconds
mysterychap@mysterychap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$ ./calculatettime.out
Enter the N value: 30000
Prime Sum: 5010567595
Normal Function Time: 0.022650 seconds
Thread Execution Time: 0.018270 seconds
mysterychap@mysterychap:/mnt/d/embedUR - Training/embedUR - assignments/advanced C-assessment/module 2$

```

Observations:

- For small values of N, the regular function runs faster than the threaded function.
- For Larger values of N, the threaded function runs faster than the regular function.

3. Know about the following topics and explore them (Write a note on your understandings)

Area for exploration,

- Child Process – fork()
- Handling Common Signals
- Time Complexity
- Locking Mechanism – Mutex and spin lock

a) Child Process – fork()

The fork() system call is the primary mechanism used in Unix- like operating systems to create a new process.

Mechanisms in fork

- Cloning: When fork() is called, the operating system creates an exact duplicate (child process) of the calling process (parent process). The child inherits clones of the parent's file descriptors, environment variables, security context, and memory state.
- Copy- on- Write (COW): To maximize efficiency, the OS does not copy the physical memory pages immediately. Instead, both parent and child share the same memory pages. Physical pages are only duplicated if either process attempts to write data, saving massive amounts of system resources.
- Return Values: The key to using fork() safely is using its integer return value to branch the code:
 - i) Negative Value: Process creation failed.
 - ii) Zero: Code is currently executing inside the newly created Child Process.
 - ii) Positive Value: Code is executing inside the Parent Process; the returned number represents the unique Process ID (PID) of the new child.

b) Handling Common Signals

Signals are asynchronous notifications sent by the operating system kernel to a process to inform it that a specific event or interruption has occurred. Some of the common signals and their uses are,

Signal	Name	Trigger Condition	Catchable / Overridable
SIGINT	Signal Interrupt	Triggered when user presses Ctrl + C in terminal.	Yes (Can write a custom exit routine).
SIGTERM	Signal Terminate	Generic request to kill program (default used by kill command).	Yes (Gives the program time to clean up memory/files).
SIGKILL	Signal Kill	Forcefully stops the process immediately.	No (Handled directly by the kernel; cannot be caught or ignored).
SIGSEGV	Segmentation Fault	Occurs if a program accesses unallocated or illegal memory.	Yes (Though difficult to safely recover from).

c) Time Complexity

Time complexity represents a theoretical metric used to quantify the amount of computational time an algorithm takes to run as a function of the input size N. It strips away physical hardware variations and measures operation scaling.

Notations:

- i) Big- O Notation: Describes the strict worst- case scenario. It acts as an absolute upper bound on design limits.
- ii) Omega Notation: Describes the best- case scenario or lower bound performance.
- iii) Theta Notation: Describes the tight bound scenario when best and worst cases scale at identical algorithmic speeds.

Commonly used complexity:

- $O(1)$ - Constant Time: Runtime remains static regardless of size (e.g., Array index lookups).
- $O(\log N)$ - Logarithmic Time: Problem size is halved at every execution step (e.g., Binary Search).
- $O(N)$ - Linear Time: Runtime scales sequentially and proportionately to input volume (e.g., Single loops, basic array parsing).
- $O(N * \log N)$ - Log-Linear Time: Standard performance for highly optimized sorting operations (e.g., Merge Sort, Quick Sort).
- $O(N^2)$ - Quadratic Time: Nested loops iterating through data sets twice (e.g., Bubble Sort)

d) Locking Mechanism – Mutex and Spin Lock

Both primitives prevent race conditions by enforcing mutual exclusion over shared critical memory structures, but their architectural approach to waiting is completely different.

1. Mutex

- A sleep-wait primitive. When a thread attempts to acquire a locked Mutex, the kernel puts that thread to sleep and initiates a context switch to allow other programs to use the CPU.
- Highly CPU efficient for long-duration lock cycles; avoids burning processor cycles while waiting.
- High CPU overhead due to context switching if the lock is held for only a fraction of a millisecond.

2. Spin Lock

- A busy-wait primitive. When a thread attempts to acquire a locked spin lock, it runs continuously in a tight `while(lock == busy)` loop, checking availability repeatedly.
- Fast reaction times, ideal for short-duration tasks because it skips the time-wasting overhead of context switching. Highly popular inside kernel development.
- Consumes 100% of a CPU core's execution capacity while spinning. If used for long locks, it drastically degrades system performance.